

C++ Ortamında Konteyner Servis Yapılandırmalarının Güvenli Round-Trip Düzenlenmesi

Safe Round-Trip Editing of Container Service Configurations in C++

Esila Şahin^{*1}, Ömer Yıldız², Mehmet Berk Kartal³, Ahmet Batuhan Canlı⁴

¹Yazılım Mühendisliği Bölümü, Fatih Sultan Mehmet Vakıf Üniversitesi, İstanbul, Türkiye

^{2,3,4}TÜBİTAK BİLGEM UEKAE, Kocaeli, Türkiye

^{*}Sorumlu yazar: esila.sahin@stu.fsm.edu.tr, ORCID: 0009-0007-8858-9540

Özet—Docker Compose dosyaları yalnızca insanlar tarafından değil, dağıtım otomasyonu tarafından da düzenlenmektedir. Böyle bir düzenleme yalnızca YAML ayrıştırmaktan ibaret değildir: dosyada aracın modellemediği alanlar, kullanıcı tanımlı `x-*` uzantıları, kısa ve uzun sözdizimi varyantları ve anlam taşıyan tırnak tercihleri bulunur; naif bir ayrıştır–değiştir–serileştir akışı hedeflenen alanın ötesinde değişiklik ya da bilgi kaybı üretir. Python ve JavaScript ekosistemlerinde biçim koruyan YAML kütüphaneleri bulunmasına karşın, C++ tarafında Compose’a özgü bir düzenleme API’si ve koruma garantisi belgelenmiş bir YAML kütüphanesi bulamadık. Bu bildiride, Compose kavramlarını yansıtan tipli bir C++ API sunan ve düzenlemeleri düğüm ağacında yerinde yapan ComposeMorph’u sunuyoruz. 647 gerçek dünya ve 38 kontrollü Compose dosyası üzerinde yedi deneyle doğruluk, koruma, değişiklik yerelliği, ileriye dönük uyumluluk ve performansı ölçtük. Bilinmeyen özelliklerin ve `x-*` uzantılarının tamamı korunmaktadır. Değerlendirme ayrıca yamll-cpp tabanlı serileştirmede sessiz bir tip değişimi kusuru ortaya çıkarmıştır: tırnaklı skalerler tırnaksız yazıldığında `docker compose config` çıktılarının yalnızca %38,1’ini kabul etmektedir. Ayrıştırıcının sakladığı tırnak bilgisini kullanan serileştirme bu oranı %100’e çıkarmakta ve ölçülebilir bir performans bedeli getirmemektedir. Tüm veri kümeleri, betikler ve ham sonuçlar tek komutla yeniden üretilebilir biçimde yayımlanmıştır.

Anahtar Kelimeler—Docker Compose; YAML round-trip düzenleme; kod olarak yapılandırma (configuration-as-code); yapı koruma; C++

Abstract—Docker Compose files are increasingly modified programmatically by deployment automation, continuous integration and delivery (CI/CD) pipelines, and configuration-as-code tooling. Treating these files as generic YAML documents is error-prone: naive parse–modify–serialize cycles discard user-defined extension fields (`x-*`), unrecognized properties introduced by newer Compose Specification versions, and configuration structure unrelated to the intended change, producing configuration churn that complicates code review. To the best of our knowledge, no existing C++ library combines a Compose-aware, class-based API with forward-compatible preservation of unknown and extension properties validated through systematic round-trip testing. We present ComposeMorph, a C++20 library that exposes typed accessors for Compose services (image, environment, ports, volumes, networks, health checks, deployment) while editing the underlying document tree in place, so that untouched nodes—including unrecognized fields and `x-*` extensions—remain structurally intact, and that writes files atomically. We evaluate ComposeMorph in seven experiments over a controlled corpus

of 38 files and a real-world corpus of 647 Compose files collected from GitHub, measuring round-trip correctness, preservation, change locality, Docker Compose CLI validation success and performance against a baseline built directly on yamll-cpp. All 802 injected unknown properties and all 1,203 `x-*` extension fields survive a targeted single-property edit, and once the baseline save noise is subtracted, edits stay confined to the single intended line for nine of ten properties. The evaluation also exposes a silent type-coercion defect in yamll-cpp-based serialization: when quoted scalars are re-emitted unquoted, `docker compose config` accepts only 38.1% of the outputs. Serialization driven by the quote information the parser already records raises this to 100% (892/892) at no measurable performance cost—in a paired in-process comparison the quote-preserving emitter is 3–21% faster. Preservation is structural rather than textual: comments and blank lines are not retained, so only 4.0% of files are byte-identical after an unmodified save. All datasets, scripts and raw results are published and reproducible with a single command.

Keywords—Docker Compose; YAML round-trip editing; configuration as code; structure preservation; C++

I. GİRİŞ

Docker Compose, çok konteynerli uygulamaların servislerini, ağlarını, depolama birimlerini ve bunlar arasındaki bağımlılıkları tek bir YAML dosyasında bildirimsel olarak tanımlamanın fiilî standardı hâline gelmiştir [1]. İbrahim ve arkadaşlarının 4.103 açık kaynak GitHub projesi üzerinde yaptığı çalışma, Compose’un yalnızca çok bileşenli sistemlerde değil, projelerin dörtte birinden fazlasında tek bileşenli uygulamalar için bile kullanıldığını göstermektedir [2]. Bu dosyalar, kaynak kodla birlikte sürüm kontrolünde tutulan, elle yazılıp bakımı yapılan birer *kod olarak yapılandırma* (configuration-as-code) varlığıdır.

Compose dosyaları giderek yalnızca insanlar tarafından değil, programlar tarafından da değiştirilmektedir. Sürekli entegrasyon ve dağıtım hatları yeni bir sürüm çıktığında servis imajının etiketini günceller; kurum içi (on-premise) kurulum ve dağıtım araçları hedef ortama göre ortam değişkenlerini, ek host kayıtlarını ya da volume kaynaklarını ayarlar. Bu çalışmayı motive eden senaryoda bir C++ dağıtım aracı, bir kriptografi servisinin imajını yeni sürüme yükseltmekte, donanım güvenlik modülüne (HSM) erişim için bir `extra_hosts`

kaydı eklemekte ve yeni sürümün çalıştırılabilir dosyalarını gösteren volume kaynağını değiştirmektedir. Bu değişikliklerin her biri kavramsal olarak dosyadaki tek bir alana dokunur.

İlk bakışta bu tür bir düzenleme basittir: dosyayı ayrıştır, ilgili alanı değiştir, geri yaz. Ancak bir Compose dosyası, onu düzenleyen programın bildiği alanlardan ibaret değildir. Dosyada kullanıcı tarafından tanımlanmış $x-*$ uzantı alanları, düzenleyici aracın henüz modellemediği daha yeni Compose Specification alanları, aynı bilginin kısa ve uzun sözdizimi (short/long syntax) gibi farklı gösterimleri, yazarın tırnak kullanımı ve düzen tercihleri bulunabilir. Bu nedenle naif bir *ayrıştır → değiştir → serileştir* akışı, hedeflenen alanın çok ötesine taşan gereksiz değişikliklere (configuration churn) ya da bilgi kaybına yol açabilir.

Üstelik bu kayıp her zaman görünür değildir. C++ tarafında akla gelen ilk yol olan `yaml-cpp` [3] tabanlı bir düzenleyici, hiçbir değişiklik yapılmadan kaydedilen bir dosyada bile `version: "3.8"` satırının tırnaklarını kaldırır. Ortaya çıkan `version: 3.8` ifadesi bir kayan noktalı sayı olarak çözümlendiği için `docker compose config` dosyayı “version must be a string” hatasıyla reddeder. Dosya sözdizimsel olarak geçerli bir YAML belgesidir ve düzenleyicinin kendisi tarafından sorunsuz okunur; hata ancak Docker dosyayı okuduğunda ortaya çıkar. Kütüphanemizin ilk sürümü de bu davranışı miras almıştı; deneylerimiz hatayı ortaya çıkarmış, kök nedenini `yaml-cpp`’nin serileştiricisinde tespit etmiş ve Bölüm V’de anlatılan tırnak koruyan serileştirme ile gidermemizi sağlamıştır.

C++ ekosistemi bu probleme karşı özellikle hazırlıksızdır. Python’da `ruamel.yaml` [4], JavaScript’te `eemeli/yaml` [5] yorumları, anahtar sırasını ve biçimi koruyan olgun round-trip desteği sunar; Go ekosisteminde ise Compose Specification’ın başvuru kütüphanesi `compose-go` [6] ve Kubernetes araçlarının kullandığı `kyaml` [7] bulunur. C++ tarafında incelediğimiz YAML kütüphanelerinin hiçbir yorum veya biçim koruyan round-trip düzenlemeyi desteklenen bir özellik olarak belgelememektedir: `yaml-cpp`’de yorumların korunmasına yönelik talep 2015’ten beri açıktır ve kütüphanenin geliştiricisi 2026’da hâlâ böyle bir seçenek bulunmadığını belirtmiştir [8], [9]. Bildiğimiz kadarıyla, Docker Compose’a özgü tip-farkında düzenlemeyi ve ölçülmüş ileriye dönük uyumlu bilinmeyen-özellik korumasını bir arada sunan bir C++ kütüphanesi bulunmamaktadır (Bölüm III).

Bu bildiride, Docker Compose yapılandırmalarının C++ uygulamalarından güvenli ve en az yan etkiyle değiştirilmesi için `ComposeMorph` adlı bir kütüphane sunuyor ve bu problemin gerçekte ne kadarının çözülebildiğini sistematik olarak ölçüyoruz. `ComposeMorph`, `yaml-cpp`’nin düğüm ağacı üzerine, Compose kavramlarını (servis, ortam değişkenleri, portlar, volume’lar, ağlar, sağlık kontrolü, dağıtım kaynakları vb.) doğrudan yansıtan sınıf tabanlı, tip-farkında bir API kurar. Her düzenleme yalnızca hedef düğümü yerinde değiştirir; kütüphanenin modellemediği alanlar hiçbir zaman yeniden inşa edilmez, bu yüzden bilinmeyen özellikler ve $x-*$ uzantı alanları dokunulmadan kalır. Kaydetme, kaynaktaki tırnaklı yazılmış skalerleri tırnaklı olarak yeniden üreten ve böylece değerlerin

YAML tiplerini koruyan bir serileştirici üzerinden yapılır. Modellenmemiş alanlar için nokta-yollu (dot-path) genel bir özellik API’si, dosya bütünlüğü için de geçici dosya üzerinden atomik kaydetme sağlanır.

Değerlendirme; doğruluk, koruma, değişiklik yerelliği, ileriye dönük uyumluluk ve performans başlıklı beş araştırma sorusu etrafında, 647 gerçek dünya GitHub Compose dosyasından oluşan bir veri kümesi ile 38 kontrollü dosyadan oluşan ikinci bir veri kümesi üzerinde yedi deneyle yapılmıştır. Gerçek dosyalara kontrollü olarak eklenen bilinmeyen özellikler ve $x-*$ uzantı alanları, ilgisiz bir alan değiştirildikten sonra tüm vakalarda korunmuştur (802/802 bilinmeyen özellik ve 1.203/1.203 uzantı alanı örneği). `docker compose config` tarafından geçerli kabul edilen girdilerden üretilen çıktıların tamamı (892/892) yeniden geçerli bulunmuştur; aynı düzenlemeleri `yaml-cpp`’nin varsayılan serileştiricisiyle yapan temel çizgide bu oran %38,1’de (340/892) kalmaktadır. Korpustaki 9.807 tırnaklı skaler değerin hiçbirisi tip değiştirmemiş, temel çizgide ise 452 dosyada 2.570 değer tip değiştirmiştir; çıktının girdiyle aynı veriye çözümlenme oranı %33’ten %100’e çıkmaktadır. Tırnak koruyan serileştirici bir performans bedeli getirmez: aynı süreçte yapılan eşleşik ölçümde `yaml-cpp`’nin kendi serileştiricisinden %3–21 daha hızlıdır (Wilcoxon işaretli sıralar testi, $p < 10^{-10}$); tek bir alanı değiştirmenin maliyeti ise ihmal edilebilir düzeydedir. Buna karşın sağlanan koruma yapısaldır, tam anlamıyla metinsel değildir: yorumlar ve boş satırlar korunmadığı için dosyaların yalnızca %4,0’ü (26/647) değişiklik yapılmadan kaydedildiğinde bayt düzeyinde aynı kalır. Ham `yaml-cpp` ile yapılan karşılaştırma ikinci bir katkıyı ortaya koyar: aynı düzenlemeleri ilgili alt ağacı yeniden inşa ederek yapan, makul görünen ham `yaml-cpp` kodu ölçülebilen hiçbir vakada kardeş alanları koruyamamıştır (0/149, 0/39, 0/116), aynı düzenlemeler `ComposeMorph`’un tip-farkında API’si üzerinden yapıldığında ise tüm vakalarda korunmuştur.

Bu bildirinin katkıları şunlardır:

- 1) Docker Compose yapılandırmalarının C++ uygulamalarından sınıf tabanlı ve tip-farkında biçimde değiştirilmesini sağlayan, modellenmemiş alanlar için genel bir özellik API’si ve atomik kaydetme içeren, yeniden kullanılabilir bir kütüphane (`ComposeMorph`).
- 2) Hedef düğümün yerinde değiştirilmesine dayanan, gelecekteki Compose Specification alanlarına karşı ileriye dönük uyumluluk sağlayan bilinmeyen-özellik ve uzantı alanı koruma mekanizması ile bu mekanizmanın gerçek dünya dosyaları üzerindeki ölçümü.
- 3) C++ YAML serileştirmesinde sessiz tip değişimine yol açan bir hata sınıfının teşhisi ve giderilmesi: `yaml-cpp`’nin serileştiricisi, tırnaklı bir skalerin taşıdığı ! etiketini yok sayarak değeri tırnaksız yazmakta, böylece string değerler sayıya veya boolean’a dönüşmekte ve dosya `docker compose config` tarafından reddedilmektedir. Etiketli dikkate alan serileştirme bu davranışı ortadan kaldırmakta, etkisi önce/sonra ölçülmektedir (geçerlilik %38,1 → %100). Aynı değerlendirme, kısa sözdizimli `environment`, `labels` ve

`extra_hosts` listelerinin düzenleme sırasında eşleme (mapping) yapısına dönüştürülmesi biçimindeki ikinci bir hatayı da ortaya çıkarmış ve gidermiştir.

- 4) İki veri kümesi (38 kontrollü ve 647 gerçek dünya Compose dosyası), yedi deney ve tüm sonuçları tek komutla yeniden üreten betiklerden oluşan, açık ve tekrarlanabilir bir değerlendirme paketi.

Bildirinin geri kalanı şöyle düzenlenmiştir. Bölüm II Docker Compose, YAML ve round-trip düzenleme problemine ilişkin altyapıyı, Bölüm III ilgili çalışmaları sunar. Bölüm IV ve Bölüm V kütüphanenin tasarımını ve round-trip dönüşüm yaklaşımını, Bölüm VI gerçekleştirimi anlatır. Bölüm VII deneysel yöntemi, Bölüm VIII bulguları verir. Bölüm IX sonuçları tartışır; Bölüm X ve Bölüm XI geçerlilik tehditlerini ve kısıtlamaları ele alır; Bölüm XII çalışmayı sonuçlandırır.

II. ALTYAPI VE MOTIVASYON

A. Docker Compose Specification

Compose Specification [1], bir uygulamanın servislerini ve bunların ağlarını, volume'larını, config ve secret tanımlarını YAML biçiminde tanımlar. Belgenin üst seviyesinde `services`, `networks`, `volumes`, `configs`, `secrets` gibi bölümler ile `x-` önekli *uzantı alanları* bulunur. Uzantı alanları spesifikasyonun araçlara ayırdığı resmî genişletme noktasıdır: şema bunları serbest bırakır, ancak `x-` öneki taşımayan tanınmayan anahtarları reddeder. Spesifikasyon ayrıca aynı bilgiyi iki biçimde ifade etmeye izin verir. Örneğin portlar kısa sözdiziminde `"8080:80"` biçiminde bir dizi elemanı, uzun sözdiziminde ise `target/published` anahtarlarını taşıyan bir eşleme (mapping) olarak yazılabilir; `environment`, `labels`, `extra_hosts` ve `volumes` için de benzer ikilikler vardır. Bir düzenleme kütüphanesi her iki biçimi de bozmadan taşımak zorundadır.

B. YAML skalerleri ve örtük tip çözümlemesi

YAML'da bir skaler değerin tipi, tırnak kullanımına bağlıdır. Tırnaksız (plain) yazılmış `3.8` bir kayan noktalı sayı, `true` bir boolean, `8080` bir tam sayı olarak çözümlenir; aynı değerler tırnaklandığında string olur. YAML 1.1 çözümleme kuralları bu kümeyi `yes/no/on/off` gibi boolean eşdeğerlerini, onaltılık ve sekizlik tam sayıları, `1:30` biçimindeki altmışlık (sexagesimal) sayıları ve tarih/zaman damgalarını da içerecek biçimde genişletir. Dolayısıyla tırnak, YAML'da biçimsel bir tercih değil, anlam taşıyan bir işaretir: `version: "3.8"` ile `version: 3.8` farklı belgelerdir ve Compose şeması `version` alanının string olmasını şart koşar.

YAML ayrıca yorum satırları, boş satırlar, girinti tercihleri, `anchor/alias` (`&ad`, `*ad`) ve birleştirme anahtarı (`<<`) gibi, belgenin veri içeriğine değil sunumuna ait öğeler barındırır. Bu öğelerin bir kısmı ayrıştırma sırasında veri modeline hiç taşınmaz.

C. Round-trip düzenleme problemi

Bir yapılandırma dosyasını programatik olarak değiştirmenin olağan yolu *ayrıştır → değiştir → serileştir* akışıdır. Bu

akışın sadakati, kullanılan ara temsile bağlıdır. Somut sözdizimi ağacı (CST) tutan araçlar kaynağın her karakterini saklar ve yalnızca dokunulan aralığı yeniden yazar; `ruamel.yaml` [4] ve `eemeli/yaml` [5] bu yaklaşımla yorumları ve biçimi korur. Buna karşılık `yaml-cpp` [3] gibi kütüphaneler belgeyi değiştirilebilir bir *düğüm ağacına* ayrıştırır: yapı, anahtar sırası ve değerler korunur, ancak sunum bilgisinin bir kısmı ayrıştırma kaybolur ve dosya kaydedilirken tüm belge yeniden üretilir. Bu, hedeflenen alanın dışındaki satırların da değişmesine (configuration churn) ve — Bölüm I'te gösterildiği gibi — değerlerin tipinin sessizce değişmesine yol açabilir.

D. İleriye dönük uyumluluk

Compose Specification gelişmeye devam etmektedir. Bir düzenleme kütüphanesinin nesne modeli, spesifikasyonun her sürümünü takip edemez. Bu nedenle kütüphanenin bilmediği alanları *kaybetmemesi*, yani modellenmemiş her yapıyı olduğu gibi taşıması pratik bir gerekliliktir. Şema temelli (dataclass/Pydantic benzeri) yaklaşımlar bu noktada dezavantajlıdır: model, tanımadığı bir alanı saklayacak bir yere sahip değilse serileştirmede o alan kaybolur.

III. İLGİLİ ÇALIŞMALAR

Bu bölüm, ayrıntılı taraması depoda yayımlanan sistematik inceleme üzerine kuruludur.¹

A. C ve C++ YAML kütüphaneleri

İncelediğimiz C/C++ YAML kütüphanelerinin hiçbir yorum veya biçim koruyan round-trip düzenlemeyi desteklenen bir özellik olarak belgelememektedir. `yaml-cpp` [3] yorumları ayrıştırma sırasında atar; bunların korunmasını isteyen talep 2015'ten beri açıktır [8] ve aynı isteği yineleyen ikinci bir kayıt 2026'da kütüphane geliştiricisi tarafından “şu anda böyle bir seçenek yok” notuyla kapatılmıştır [9]. `libyaml` [10] ayrıştırıcısı yorum belirteci hiç üretmez; bu, eksik bir kolaylık API'si değil mimari bir sınırdır [11]. `rapidyaml` [12] kaynağı kopyalanmayan görünüm tabanlı bir tasarıma sahiptir ve yorum koruma iddiası taşımaz; serileştiricisinde skaler tip sadakatini bozan bir hata 2020'de bildirilmiş ve düzeltilmiştir [13]. Bu kayıt, bu bildiride ölçtüğümüz hata sınıfının C++ serileştiricilerinde daha önce de görüldüğünü göstermesi bakımından anlamlıdır. `fkYAML` [14] başlık-tabanlı ve hızlıdır, ancak belgelerinde round-trip koruma iddiası yoktur.

B. Yorum koruyan ve CST tabanlı araçlar

Python'da `ruamel.yaml` [4] yorumları, anahtar sırasını ve akış biçimini *yükle-değiştir-yaz* döngüsü boyunca korur. JavaScript'te `eemeli/yaml` [5] aynı yeteneği, kaynağın her karakterini saklayan bir CST katmanı üzerinden sunar. Rust tarafında `yaml-edit` [15], kayıpsız sözdizimi ağaçları için kullanılan `rowan` altyapısına dayanarak biçim, yorum ve boşlukları korur; benzer kayıpsız-ağaç deseni Python kaynak kodu için `LibCST` [16] ve genel amaçlı `cstree` [17] ile de kullanılmaktadır. Bu araçların tümü genel amaçlı YAML'a yöneliktir; hiçbir

¹`docs/related-work.md`; issue durumları 11 Eylül 2026'da GitHub API üzerinden doğrulanmıştır.

Compose'a özgü bir nesne modeli sunmaz ve hiçbiri C++ değildir.

C. Kubernetes ve Docker Compose araçları

Kubernetes ekosisteminde kyaml [7], manifest dosyalarını dönüştürürken yapıyı ve yorumları korumayı hedefler; kustomize ve kpt bu kütüphane üzerine kuruludur. Belgelemiş sınırı, altında kullandığı go-yaml'ın yorumları her zaman doğru işleyememesi ve bazı yorumların yanlış biçimlenmesi ya da kaybolmasıdır [18]. Docker Compose tarafında compose-go [6] spesifikasyonun resmî başvuru kütüphanesidir, ancak bir yükleyici/doğrulayıcıdır; koruma odaklı bir düzenleme API'si sunmaz. Diğer dillerde Compose dosyasını programatik olarak değiştiren kütüphaneler mevcuttur: Go'da fishfinger [19], Python'da Pydantic/dataclass modelleri üzerinden çalışan compose-py [20], JavaScript'te zincirlenebilir tipli bir API sunan ve bu yönüyle ComposeMorph'un Service API'sine en çok benzeyen docker-compose-yaml-parser [21]. Bunların hiçbiri bilinmeyen alan veya x-* koruma oranını ölçülmüş bir garanti olarak belgelemez.

D. Ampirik çalışmalar

Ibrahim ve arkadaşları 4.103 GitHub projesinde Compose kullanımını incelemiş, dosyaların büyük bölümünün küçük ve seyrek değiştirilen yapılandırmalar olduğunu raporlamıştır [2]; Eng ve arkadaşları gerçek projelerden çok konteynerli kompozisyon örüntülerini kataloglamıştır [22]. Bu çalışmalar, GitHub kaynaklı Compose korpuslarının bu tür değerlendirmeler için yerleşik bir veri kaynağı olduğunu göstermesi bakımından yöntemimizi destekler. Daha geniş çerçevede, YAML'ın model yönetimi dilleriyle işlenmesi üzerine yapılan çalışmalar [23] dönüşümü model seviyesinde ele alır; bu bildirinin odağı ise mevcut bir dosyanın en az yan etkiyle düzenlenmesidir.

E. Konumlandırma

Bildiğimiz kadarıyla, Docker Compose'a özgü tip-farkında düzenlemeyi ve ölçülmüş ileriye dönük uyumlu bilinmeyen-özellik korumasını bir arada sunan bir C++ kütüphanesi bulunmamaktadır. Bu ifadeyi, taramamızın desteklediği sınırlar içinde kullanıyoruz: ComposeMorph yorum veya tam biçim koruma konusunda diğer ekosistemlerdeki olgun araçlardan daha iyi değildir ve bunu amaçlamaz (Bölüm XI).

IV. TASARIM

A. Nesne modeli

Kütüphanenin giriş noktası ComposeFile sınıfıdır: bir dosyayı yükler, servislere ve üst seviye koleksiyonlara erişim verir, dosyayı kaydeder. ComposeFile::service(ad) çağrısı bir Service nesnesi döndürür; Service de Compose kavramlarını doğrudan yansıtan alt nesneler sunar: Environment (ortam değişkenleri ve labels), Ports, Volumes, Networks, ExtraHosts, BuildConfig, HealthCheck, DependsOn ve DeployConfig. Üst seviyede ayrıca NetworkCollection, VolumeCollection, ConfigCollection ve SecretCollection bulunur.

Bu sınıfların hiçbiri veriyi kendi içinde kopyalamaz; her biri, altta yatan YAML düğüm ağacının ilgili düğümüne tutunan ince bir görünümdür. Bir Service nesnesi, servisin düğümüne bir tutamaçtır; üzerinde yapılan her değişiklik doğrudan ağaçta gerçekleşir. Böylece kütüphanenin modellemediği kardeş alanlar, düzenleme sırasında hiçbir zaman okunup yeniden yazılmaz.

B. Tipli API ile genel API'nin ilişkisi

Tipli API, spesifikasyonun sık kullanılan alanlarını adlandırılmış işlemlerle sunar; örneğin bir imaj etiketini güncellemek tek satırdır:

```
compose.service("api")
.setImage("company/api:2.0");
```

Modellenmemiş her alan için aynı nesne, nokta-yollu genel bir özellik API'si sağlar:

```
api.set("x-company-security.hsm",
"enabled");
auto v = api.get<std::string>(
"logging.options.max-file");
api.remove("deploy.placement.constraints");
```

İki API aynı ağaç üzerinde çalışır ve birbirinin yerine geçebilir: tipli işlemler, genel API'nin yapacağı düğüm yazma işleminin adlandırılmış ve tip kontrollü bir kısayoludur. Bu ilişki, ileriye dönük uyumluluğun kaynağıdır. Spesifikasyona yarın eklenecek bir alan için kütüphanenin güncellenmesi gerekmez; alan genel API ile okunup yazılabilir, hiç doku-nulmasa bile dosyada aynen kalır.

C. Kısa ve uzun sözdizimi

Environment, ExtraHosts ve Volumes gibi sınıflar, altlarındaki düğümün liste mi yoksa eşleme mi olduğunu çalışma zamanında tespit eder ve dosyanın mevcut biçimini korur. Örneğin environment kısa sözdiziminde (- KEY=value girdileri) yazılmışsa, environment().set() çağrısı ilgili girdiyi yerinde günceller ya da listenin sonuna yeni bir girdi ekler; bölümü eşlemeye dönüştürmez. Ports ve Networks yardımcılarında bu dönüşüm henüz eksiktir (Bölüm XI).

D. Hata yönetimi ve güvenli kaydetme

Bulunamayan servis, ayrıştırılamayan dosya ve şema ihlali gibi durumlar ayrı istisna türleriyle (ParseException, ServiceNotFoundException, ValidationException) bildirilir. Kaydetme varsayılan olarak atomiktir: içerik önce geçici bir dosyaya yazılır, ardından hedefin üzerine rename ile taşınır; böylece yazma sırasında oluşan bir kesinti mevcut dosyayı yarım bırakmaz. İsteğe bağlı olarak .bak yedeği alınabilir.

V. ROUND-TRIP DÖNÜŞÜM YAKLAŞIMI

A. Ayrıştırma modeli ve neyin korunduğu

ComposeMorph, belgeyi yaml-cpp'nin değiştirilebilir düğüm ağacına ayrıştırır. Bu bir CST'dir: kaynak metnin karakter aralıkları saklanmaz. Ağaçta korunan bilgi şunlardır: belge yapısı, eşlemelerde anahtar sırası, koleksiyonların akış/blok stili, skaler değerlerin metni, anchor ile paylaşılan

düğümün kimliği ve — bu çalışma için kritik olan — skalerin kaynağa tırnaklı yazılıp yazılmadığını gösteren *belirtisiz etiket*: tırnaklı skalerler !, tırnaksız (plain) skalerler ? etiketi taşır. Ayırıştırma kaybolan bilgi ise yorumlar, boş satırlar, girinti genişliği, tek/çift tırnak ayrımı ve anchor adlarıdır.

B. Değişiklik izleme: yok

Kütüphane bir değişiklik günlüğü tutmaz. Her düzenleme, ilgili düğüme doğrudan yazar; dolayısıyla birleştirilecek iki sürüm ve çözülecek bir çakışma durumu oluşmaz. Modellenmemiş alanlar hiçbir aşamada okunmaz, kopyalanmaz veya yeniden inşa edilmez; kaydetmede ağaçta ne varsa o yazılır. Bilinmeyen özelliklerin ve x-* alanlarının korunması, ek bir mekanizmanın değil bu tasarım kararının sonucudur.

C. Serileştirme

yaml-cpp'nin varsayılan serileştiricisi bir skaleri tırnaksız yazmanın güvenli olup olmadığına yalnızca metnine bakarak karar verir: 0.8.0 sürümünde `IsValidPlainScalar` işlevi değeri null benzeri dizgilere (null, ~, boş dizgi) ve sözdizimsel olarak sakıncalı karakterlere karşı sınırlı, fakat boolean veya sayı gibi görünmesini dikkate almaz. Ayrıca ayırıştırma üretilen ! etiketi serileştirmede yok sayılır. Sonuç, Bölüm I'teki davranıştır: `version: "3.8"` tırnaksız yazılır ve yeniden okunduğunda artık string değil kayan noktalı sayıdır.

Bu nedenle `ComposeMorph`, kaydetmeyi kendi ağaç yürütücüsü üzerinden yapar. Yürütücü, yaml-cpp'nin kendi düğüm serileştiricisiyle aynı çıktıyı üretir (aynı anchor/alias numaralandırması, aynı etiketler, aynı akış/blok stili, aynı anahtar sırası); tek farkı skaler yazma kararıdır:

- 1) **Birinci geçiş.** Ağaç dolaşarak her düğümün kaç kez referans edildiği sayılır. Bir düğüme birden çok yoldan ulaşıyorsa o düğüm anchor gerektiriyor demektir. Kimlik karşılaştırması düğüm kimliği üzerinden yapılır; arama uzayını daraltmak için düğümler kaynağındaki konumlarına göre kovalara ayrılır.
- 2) **İkinci geçiş.** Ağaç yeniden dolaşılır ve yazılır. Birden çok referanslı bir düğüme ilk ulaşıldığında bir anchor atanır, sonraki ulaşımarda alias yazılır. Eşlemeler ve diziler için düğümün stili (akış/blok) uygulanır.
- 3) **Skaler yazma kuralı.** Düğüm ! etiketi taşıyorsa (yani kaynağa tırnaklıydysa) değer çift tırnakla yazılır; aksi hâlde yazma kararı yaml-cpp'nin varsayılan davranışına bırakılır. Böylece kaynağa tırnaksız yazılmış sayılar ve boolean'lar tırnaksız kalır, tırnaklı yazılmış string'ler string olarak kalır.

D. API üzerinden yazılan değerler

Round-trip sadakati, yalnızca dosyadan gelen değerleri kapsar. API üzerinden yazılan bir string de aynı tuzağa düşebilir: örneğin `setCommand({"run", "-workers", "4"})` çağrısında "4" tırnaksız yazılırsa `Compose` şeması `command` dizisinin tüm elemanlarının string olmasını beklediği için dosya reddedilir. Bu nedenle string alan her setter, yazdığı değeri bir belirsizlik sınavından geçirir: değer tırnaksız yazıldığında YAML 1.1 veya YAML 1.2 çekirdek şemasına göre

null, boolean, tam sayı, kayan nokta, altmışlık sayı, zaman damgası ya da birleştirme anahtarı olarak çözümlenecekse düğüm ! ile işaretlenir ve yukarıdaki kural gereği tırnaklı yazılır. Sınama kasıtlı olarak geniş tutulmuştur: gereksiz tırnaklamanın bedeli estetik, eksik tırnaklamanın bedeli sessiz tip değişimidir.

Mevcut bir değerin üzerine yazıldığında etiket korunduğu için, kaynağa tırnaklı olan bir alan güncellendikten sonra da tırnaklı kalır; kaynağa tırnaksız olan bir alan ise değeri belirsiz olmadıkça tırnaksız kalır.

E. Desteklenmeyen YAML özellikleri

Yorumlar ve boş satırlar ayırıştırma kaybolduğu için çıktıda bulunmaz. Anchor/alias yapıları korunur — somut değerlere genişletilmez — ancak anchor adları ayırıştırıcı tarafından saklanmadığından çıktıda otomatik üretilmiş sayısal adlarla (&1, *1) yeniden yazılır. Tek tırnaklı skalerler çift tırnakla yazılır. Bu davranışların hepsi Bölüm VIII ve Bölüm XI'da ölçülmüş biçimde raporlanmaktadır.

VI. GERÇEKLEŞTİRİM

`ComposeMorph C++20` ile yazılmıştır ve GCC 11 veya Clang 13 ve üzeri derleyicilerle kurulabilir; değerlendirmede GCC 14.2 kullanılmıştır. Derleme sistemi `CMake` (≥ 3.16), tek dış bağımlılık yaml-cpp'dir ($\geq 0.7.0$; ölçümlerde 0.8.0). Birim testleri `GoogleTest` ile yazılmış ve `CTest`'e bağlanmıştır; paket şu anda 11 test içermektedir ve bunların dördü bu çalışmada ortaya çıkan hatalar için yazılmış gerileme (regression) testleridir.

Kütüphanenin kendisi yaklaşık 2.100 satırdır (1.495 satır kaynak, 606 satır başlık). Değerlendirme altyapısı ayrıca beş adet C++ ölçüm aracı (toplam 557 satır) ve sekiz Python betiği (toplam yaklaşık 3.970 satır) içerir. Depo, görev tanımındaki yapıyı izler: `src/` ve `include/` kütüphaneyi, `benchmarks/` ölçüm araçlarını, `datasets/` iki veri kümesini, `scripts/` deney sürücülerini, `results/` ham çıktıları, tabloları ve figürleri, `paper/` ise bu bildiriye barındırır.

`Docker Compose` ile bütünleşme iki düzeydedir. Kütüphane içinde `ComposeFile::validate()` temel iş kuralı denetimleri yapar (örneğin bir servisin image ya da build alanlarından en az birini taşıması). Asıl doğrulama ise dış araçla yapılır: üretilen her çıktı `docker compose config` komutundan geçirilir (Bölüm VII).

VII. DENEYSEL YÖNTEM

A. Araştırma soruları

- **RQ1 (Doğruluk):** Kütüphane gerçek `Compose` dosyalarını ayırıştırıp değiştirdikten sonra geçerli `Compose` dosyaları üretiyor mu?
- **RQ2 (Koruma):** Yalnızca belirli bir alan değiştirildiğinde, değişiklik ile ilgisi olmayan yapılandırma bilgisinin ne kadarı korunuyor?
- **RQ3 (Değişiklik yerelliği):** Hedeflenen alan değiştirilken dosyada ne kadar gereksiz değişiklik oluşuyor?

- **RQ4 (İleriye dönük uyumluluk):** Nesne modelinde tanımlanmamış Compose özellikleri kaybedilmeden taşıyor mu?
- **RQ5 (Performans):** Yükleme, değiştirme ve serileştirme maliyeti gerçek kullanım için kabul edilebilir mi?

B. Veri kümeleri

Veri Kümesi A (kontrollü), elle yazılmış 38 dosyadan oluşur: 24 dosya Compose özelliklerini tek tek (build, kısa ve uzun sözdizimli environment/ports/volumes, networks, depends_on, healthcheck, deploy, configs, secrets, labels, profiles, logging, sysctls, ulimits, devices, x-* uzantıları, bilinmeyen özellikler ve iç içe genel özellikler) izole eder; 2 dosya bunları birlikte içerir; 10 dosya sınır durumlarını (anchor/alias ve birleştirme anahtarı, tırnaklı skaler biçimleri, yalnızca build içeren servis, Unicode içerik, null ve boş değerler, external kaynaklar, extends, include, yalnızca büyük/küçük harfle ayrılan anahtarlar) sınar; 2 dosya ise koruma deneylerinin işaretçi kalıplarını taşır.

Veri Kümesi B (gerçek dünya), açık kaynak GitHub projelerinden otomatik olarak toplanmış 647 Compose dosyasından oluşur. Toplama ve analiz betikleri depoda yayımlanmıştır. Korpus, literatürdeki gözlemlerle uyumlu biçimde küçük dosyalara eğilimlidir: satır sayısı medyanı 39 (en büyük dosya 2.591 satır), servis sayısı medyanı 2 (en fazla 56), üst seviye bölüm sayısı medyanı 2, YAML iç içelik derinliği medyanı 4 ve uzantı alanı sayısı medyanı 0'dır. Son değer, uzantı ve bilinmeyen alan deneylerinde neden kontrollü işaretçi enjeksiyonuna başvurduğumuzu açıklar: bu alanlar gerçek dosyalarda doğal olarak nadirdir.

C. Deneyler

Yedi deney yürütülmüştür: (1) kimlik round-trip (değişiklik yapmadan yükle-kaydet), (2) hedefli değişiklik ve değişiklik yerelliği (on ayrı özellik üzerinde), (3–4) bilinmeyen özellik ve x-* uzantı koruma, (5) docker compose config ile doğrulama, (6) ham yaml-cpp ile karşılaştırma, (7) performans. Her deney her iki veri kümesi üzerinde de koşulmuştur. Deney 6'da üç araç karşılaştırılır: ComposeMorph; düğümü yerinde değiştiren *dikkatli* ham yaml-cpp kodu; ve aynı düzenlemeyi ilgili alt ağacı yeniden inşa ederek yapan *naif* ham yaml-cpp kodu. Dikkatli araç yaml-cpp'nin varsayılan serileştiricisini kullandığından, Deney 5 ve tırnak analizinde aynı zamanda "düzeltme öncesi" sütununu sağlar; bu eşdeğerlik ölçülmüştür (Bölüm VIII).

D. Metrikler ve istatistiksel değerlendirme

Metinsel koruma, girdi ile çıktı arasındaki değişen satır sayısı, değişen satır oranı, satır düzeyinde Levenshtein uzaklığı ve normalize edilmiş düzenleme uzaklığı ile ölçülür. Değişiklik yerelliği için *beklenen değişen satır / gerçekleşen değişen satır* oranı kullanılır; beklenen değer tek alanlık bir düzenleme için 1'dir. Aynı dosyanın değişiklik yapılmadan kaydedilmesindeki taban gürültü, düzeltilmiş oranda gerçekleşen satır sayısından düşülür. Anlamsal koruma iki bağımsız oracle ile

TABLO I
DOCKER COMPOSE DOĞRULAMA BAŞARI ORANI (VERİ KÜMESİ B)

Çıktı	ComposeMorph	Temel çizgi
Kimlik round-trip	491/491 (%100)	187/491 (%38,1)
Hedefli image düzenlemesi	401/401 (%100)	153/401 (%38,2)
Toplam	892/892 (%100)	340/892 (%38,1)

ölçülür: docker compose config (Compose şemasının kendi doğrulayıcısı) ve PyYAML ile yeniden yükleme (değerlerin YAML tiplerinin korunup korunmadığı). Performans ölçümleri her dosya için 30 yinelemeyle yapılır; medyan, ortalama, standart sapma ve p95 raporlanır. İki serileştirici karşılaştırılırken ölçümler aynı süreçte, aynı ağaç üzerinde ve yineleme başına sıra değiştirerek eşleşik biçimde alınır; dağılım varsayımı yapmamak için Wilcoxon işaretli sıralar testi (normal yaklaşım) kullanılır.

E. Ortam ve tekrarlanabilirlik

Ölçümler 12. nesil Intel Core i7-1255U (12 mantıksal çekirdek), 15,3 GiB RAM, Debian GNU/Linux 13 (çekirdek 6.12), GCC 14.2.0, Release derleme, C++20 ve Docker Compose 2.26.1 ortamında yapılmıştır. Tüm deneyler depodaki tek bir komutla yeniden üretilebilir: `./scripts/run-all-experiments.sh`. Ham çıktılar, tablolar ve figürler `results/` altında yayımlanmıştır; düzeltme öncesi tablolar karşılaştırma için `results/pre-fix/` altında saklanmıştır.

VIII. BULGULAR

Bu bölümdeki tüm sayılar `results/tables/` altındaki üretilmiş özetlerden alınmıştır. "Temel çizgi" sütunları, aynı düzenlemeleri `yaml-cpp`'nin varsayılan serileştiricisiyle yapan dikkatli ham `yaml-cpp` aracına aittir; bu araç düzeltme öncesi ComposeMorph ile aynı çıktıyı üretmektedir (100/100 dosyada bayt düzeyinde özdeş).

A. RQ1 — Doğruluk

Veri Kümesi B'deki 647 dosyanın tamamı ayrıştırılmış (647/647), kaydedilmiş ve kaydedilen çıktı yeniden ayrıştırılabilmiştir (647/647). Veri Kümesi A'da da 38/38 dosya için aynı sonuç alınmıştır.

Asıl ayırım dış doğrulamada ortaya çıkar. Korpustaki 647 dosyanın 491'i (%75,9) `docker compose config` tarafından zaten geçerli kabul edilmektedir; kalanlar eksik `.env` dosyaları, tanımsız interpolasyon değişkenleri veya bozuk sözdizimi gibi kütüphaneden bağımsız nedenlerle reddedilir ve aşağıdaki oranların dışında tutulur. Tablo I, bu geçerli girdilerden üretilen çıktıların doğrulama sonuçlarını göstermektedir.

Temel çizgide geçersizleşen 552 çıktının 548'i, tırnaklı bir skalerin tırnaksız yazılması sonucu string olması gereken bir alanın sayıya veya boolean'a dönüşmesinden kaynaklanmaktadır. Kalan 4 vakada çıktı, Docker'ın YAML ayrıştırıcısı için sözdizimsel olarak dahi geçerli değildir: kaynakta `" : 80 "` biçiminde tırnaklanmış bir değer, akış dizisi içinde tırnaksız

yazıldığında PyYAML ve go-yaml tarafından okunamamaktadır. ComposeMorph'un tırnak koruyan serileştiricisiyle bu vakaların tamamı ortadan kalkmıştır. Veri Kümesi A'da da sonuç aynıdır (64/64'e karşı 62/64).

B. RQ2 — Koruma

Koruma üç ayrı düzeyde ölçülmüştür.

Yapısal koruma. Gerçek dosyalara kontrollü olarak eklenen ve kütüphanenin modellemediği alanlar, ilgisiz bir alan değiştirildikten sonra tüm vakalarda korunmuştur: 802/802 bilinmeyen özellik ve 1.203/1.203 $x-*$ uzantı alanı örneği (Veri Kümesi A'da 96/96 ve 144/144). Kontrollü sınır durumlarının altısı da (null değerli $x-*$, skaler değerli $x-*$, liste değerli $x-*$, liste biçiminde bilinmeyen üst seviye bölüm, tipli bir bölümün içine gömülü bilinmeyen alan, başka bir bilinmeyen bloğun içine iki düzey gömülü $x-*$) korunmuştur.

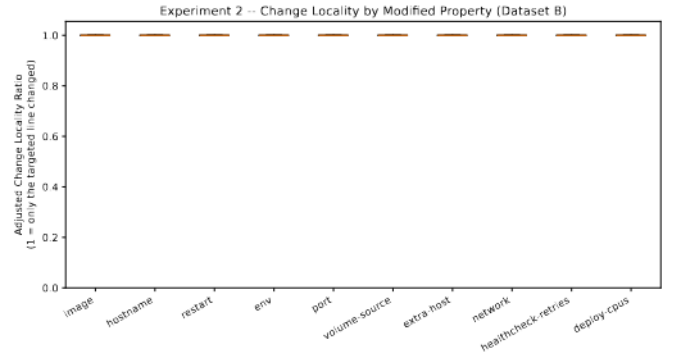
Değer tipleri. Korpustaki 617 dosyada açıkça tırnaklanmış 9.807 skaler değer tespit edilmiştir. ComposeMorph ile round-trip sonrasında bunların hiçbirisi tip değiştirmemiştir. Temel çizgide ise 452 dosyada (tırnaklı skaler içeren dosyaların %73,3'ü) 2.570 değer tip değiştirmektedir: tam sayı görünümlü değerlerin %99,9'u, kayan nokta görünümlülerin %100'ü ve boolean kelimelerinin %99,5'i. Buna bağlı olarak, çıktının girdiyle PyYAML'a göre birebir aynı veriye çözümleme oranı 33/100'den 100/100'e çıkmaktadır.

Metinsel koruma. Buna karşılık çıktı bayt düzeyinde girdinin aynısı değildir: değişiklik yapılmadan kaydedilen dosyaların yalnızca %4,0'ü (26/647) özdeştir; değişen satır oranının medyanı 0,167'dir (düzeltme öncesi %0,77 ve 0,250). Kalan farklar yorumların ve boş satırların kaybindan, girinti normalizasyonundan ve tek tırnağın çift tırnağa dönüşmesinden kaynaklanır. Anchor/alias ve birleştirme anahtarları yapıları somut değerlere genişletilmeden korunur, ancak anchor adları sayısal adlarla yeniden yazılır.

C. RQ3 — Değişiklik Yerelliği

On ayrı özellik üzerinde 1.467 düzenleme senaryosu konuşulmuştur. Ham yerellik oranının medyanı özelliğe göre 0,03–0,13 arasındadır; bu düşük değerler düzenlemenin kendisinden değil, kaydetmede yorum ve boş satır kaybindan doğan taban gürültüden kaynaklanır. Aynı dosyanın değişiklik yapılmadan kaydedilmesindeki gürültü düştüğünde, on özelliğin dokuzunda düzeltilmiş yerellik oranının medyanı 1,0'dır: düzenleme gerçekten de tek satırla sınırlı kalmaktadır. Tek istisna `extra_hosts`'a girdi eklemedir (medyan 0,5). Şekil 1 dağılımı göstermektedir.

Deneyde 32 başarısızlık gözlenmiştir ve bunların tamamı kütüphanenin bilinen bir sınırından kaynaklanır: `Networks::add` hedef düğümün dizi olup olmadığını denetlemeden ekleme yapar, bu yüzden uzun sözdizimli (eşleme) `networks` tanımlarında hata verir (31 vaka); benzer biçimde `Ports::has` liste elemanlarının string olduğunu varsayar (1 vaka). Bu sınırlar Bölüm XI'da açıkça raporlanmaktadır.



Şekil 1. Özelliğe göre düzeltilmiş değişiklik yerelliği oranı (Veri Kümesi B). 1,0 değeri, düzenlemenin yalnızca hedeflenen satırı etkilediği anlamına gelir.

TABLO II
YÜKLE-DEĞİŞTİR-KAYDET SÜRELERİ, MEDYAN (VERİ KÜMESİ B, DOSYA BAŞINA 30 YİNELEME)

Grup	Yükle	Değiştir	Kaydet	Toplam	Süreler
Küçük (<100 satır)	0,29	0,006	0,31	0,61	
Orta (100–500)	1,31	0,015	1,12	2,46	
Büyük (500–2000)	4,58	0,038	3,57	8,28	
Çok büyük (>2000)	15,83	0,139	10,85	27,17	

milisaniyedir.

D. RQ4 — İleriye Dönük Uyumluluk

Kütüphanenin nesne modelinde bulunmayan alanlar, hem $x-$ önekli uzantılar hem de gelecekteki spesifikasyon alanlarını temsil eden $x-$ öneksiz bilinmeyen özellikler için tam olarak korunmuştur (RQ2). Ancak koruma ile geçerlilik aynı şey değildir. Yalnızca $x-*$ alanları eklenen dosyaların tamamı `docker compose config` tarafından kabul edilirken (83/83; temel çizgide 33/83), $x-$ öneksiz bilinmeyen alan eklenen dosyaların hiçbirisi kabul edilmemiştir (0/83). Bunun nedeni kütüphane değil, Compose şemasının tanınmayan anahtarlarına yalnızca $x-$ öneki taşıdıklarında izin vermesidir. Pratik sonuç şudur: ileriye dönük uyumluluk için ayrılmış resmî mekanizma $x-*$ uzantılarıdır ve kütüphane bu mekanizmayı eksiksiz desteklemektedir.

E. RQ5 — Performans

Tablo II, dosya boyutuna göre yük-le-değiştir-kaydet sürelerinin medyanlarını vermektedir. Değiştirme adımının maliyeti diğer iki adımın yanında ihmal edilebilir düzeydedir; toplam süre en büyük dosyalarda bile onlarca milisaniye mertebesinde. Tepe bellek kullanımı 4,7 MiB ile 8,2 MiB arasındadır.

Tırnak koruyan serileştiricinin bir maliyeti olup olmadığını ölçmek için iki serileştirici aynı süreçte, aynı ağaç üzerinde ve yineleme başına sıra değiştirilerek karşılaştırılmıştır. ComposeMorph'un serileştiricisi tüm boyut gruplarında `yamll-cpp`'nin kendi serileştiricisinden hızlıdır: medyan oran küçük dosyalarda 0,97, orta ve büyük dosyalarda 0,94, çok büyük dosyalarda 0,79'dur ve fark her grupta istatistiksel olarak anlamlıdır (Wilcoxon işaretli sıralar testi, $p < 10^{-10}$). Tek

ek maliyet, API üzerinden yazılan değerlerin belirsizlik sınımasında kullanılan düzenli ifadenin süreç başına bir kez derlenmesidir (yaklaşık 1,7 ms); bu maliyet ölçümlerde ilk yinelemeye düşer ve ortalama ile p95 değerlerini yukarı çeker.

F. Ham yaml-cpp ile karşılaştırma

Deney 6, ComposeMorph'u iki ham yaml-cpp kullanımıyla karşılaştırır. Serileştirme düzeyinde ComposeMorph ile dikkatli ham kullanım arasındaki çıktı farkı 100 dosyanın 96'sında yalnızca tırnak karakterlerinden ibarettir, 4'ünde çıktı zaten özdeştir; tırnak dışında farklılık yoktur. Düzenleme güvenliği düzeyinde ise fark büyüktür (Şekil 2): alt ağacı yeniden inşa eden naif kullanım, ölçülebilen hiçbir vakada düzenlenen servisin diğer alanlarını koruyamamıştır (image için 0/149, hostname için 0/39, env için 0/116), ComposeMorph ise tümünü korumuştur (149/149, 39/39, 116/116). Aynı desen işaretçi alanlarında da görülür: naif kullanım servis düzeyindeki 200 işaretçinin tamamını kaybederken üst seviyedekilerin tamamını korur; hasar, üzerine yazılan alt ağaçla sınırlıdır.

Ergonomi açısından, ComposeMorph'ta on düzenlemenin her biri tek satırdır; aynı düzenlemeleri ham yaml-cpp ile doğru yapmak 1 ile 24 satır arasında kod gerektirir (volume-source için 24, extra-host için 22, env için 18 satır). Buna karşılık naif ve hatalı sürüm image için 5, hostname için 3 satırdır: yanlış kod, doğru koddan daha fazla emek gerektirmemektedir.

IX. TARTIŞMA

A. Tırnak, biçim değil anlamdır

Bu çalışmanın en aktarılabılır bulgusu, düğüm ağacı tabanlı bir YAML kütüphanesinde tırnak kaybının kozmetik bir sorun olarak görülmesinin yanıltıcı olmasıdır. Ölçümlerimizde bu davranış, geçerli girdilerden üretilen çıktıların %62'sinin Docker tarafından reddedilmesine yol açmıştır. Kusurun kaynağı, ayrıştırıcının tırnak bilgisini aslında saklaması (! belirtisiz etiketi), ancak serileştiricinin bu bilgiyi yok sayarak kararını yalnızca değer metnine dayandırmasıdır. Düzeltme, yeni bir ara temsil gerektirmez: mevcut etiketin okunması yeterlidir. Bu nedenle aynı teknik, yaml-cpp üzerine kurulu diğer yapılandırma araçlarına da doğrudan uygulanabilir.

Bulgunun ikinci yüzü, hangi oracle ile bakıldığının sonucu değiştirmesidir. PyYAML'ın YAML 1.1 çözümleme kurallarına göre tip değiştiren her değer, Docker'ın kullandığı goyaml için sorun oluşturmaz; örneğin altmışlık sayı biçimi goyaml'da desteklenmez. Buna karşılık version ya da command elemanları gibi şemanın string dayattığı alanlarda hata kesindir. Bu yüzden ölçümü tek bir doğrulayıcıya bağlamak yerine iki bağımsız oracle kullandık.

B. Hangi senaryoda ne beklenmeli

Kütüphane, otomasyonun dosyaya tek bir alanla dokunduğu senaryolar için uygundur: sürüm yükseltme, ortam değişkeni ayarlama, volume kaynağı değiştirme. Bu senaryolarda düzenleme, taban gürültü çıkarıldığında gerçekten tek satırda kalmakta, bilinmeyen ve x-* alanlar korunmakta ve çıktı

Docker tarafından kabul edilmektedir. Buna karşılık dosyanın insan tarafından okunan biçimini — yorumlarını, boş satırlarını, girintisini — olduğu gibi korumak gerekiyorsa, bu kütüphane uygun araç değildir; bunun için CST tabanlı çözümler (Python'da ruamel.yaml, JavaScript'te eemeli/yaml) gerekir. Kod incelemesinde yalnızca değiştirilen satırın görünmesi isteniyorsa, mevcut durumda ortaya çıkan diff bu beklentiye karşılamaz.

C. Performans ile koruma arasında bir ödünleşme yok

Beklenenin aksine, koruma için ödenen bir performans bedeli ölçülmemiştir. Tırnak koruyan serileştirici, yaml-cpp'nin kendi serileştiricisinden her boyut grubunda daha hızlıdır; bunun nedeni büyük olasılıkla yaml-cpp'nin düğüm serileştirmesini olay tabanlı bir ara katman üzerinden yapmasıdır. Düzenleme adımının maliyeti ise yükleme ve kaydetmenin yanında ölçülemeyecek kadar küçüktür. Dolayısıyla pratikte maliyet, dosyanın ayrıştırılması ve yeniden yazılmasıdır; bu da onlarca milisaniyeyi aşmaz.

D. Compose'a özgü API neden fark yaratıyor?

Deney 6, farkın nerede olduğunu net biçimde ayırır. Doğru yazılmış ham yaml-cpp kodu, ComposeMorph ile aynı koruma davranışını üretebilir; yani koruma, sihirli bir algoritmanın değil, doğru düzenleme tekniğinin sonucudur. Ancak bu tekniği her çağrı yerinde doğru uygulamak programcının sorumluluğundadır ve yanlış uygulamak daha fazla emek gerektirmez: alt ağacı yeniden inşa eden sürüm hem daha kısadır hem de derlenir; kaybı yalnızca çalışma zamanında, üstelik sessizce görülür. Tipli API'nin katkısı, doğru olanı varsayılan yol hâline getirmesidir. Aynı gözlem kısa sözdizimi hatasında da doğrulanmıştır: liste biçimli environment bölümünü eşlemeye çeviren davranış, ham kullanımda da kütüphanenin ilk sürümünde de aynı biçimde ortaya çıkmıştır.

X. GEÇERLİLİK TEHDİTLERİ

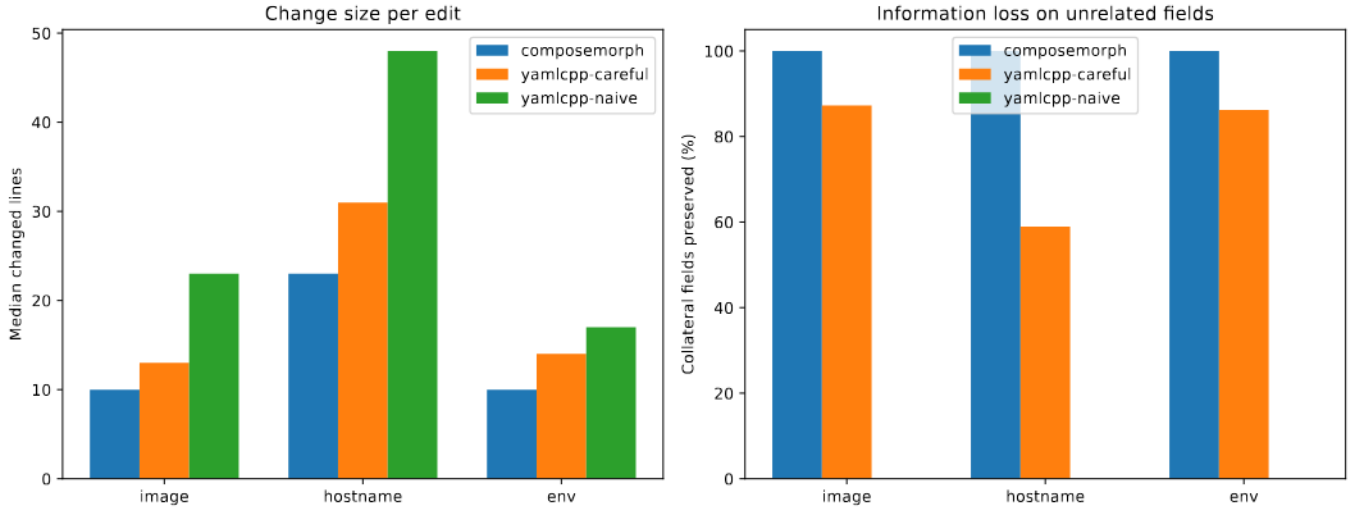
A. İç Geçerlilik

Deney altyapısı bizim tarafımızdan yazılmıştır ve kendi hatalarını ölçümlere taşıyabilir. Nitekim ilk koşullarda, imaj etiketini son iki nokta işaretinden ayıran ölçüm kodu, \${VAR:-default} biçiminde interpolasyon içeren imaj adlarını bozmuş ve 11 dosyada kütüphaneye ait olmayan bir hata üretmişti; bu, doğrulama çıktılarında ayrı bir kategori olarak görünmüş ve düzeltilmiştir. Koruma deneylerinde kullanılan bilinmeyen ve x-* alanlar sentetiktir; gerçek dosyalarda bu alanların nadir olması (uzantı alanı sayısı medyanı 0) nedeniyle enjeksiyona başvurulmuştur, dolayısıyla ölçülen koruma oranı bu kalıplar için geçerlidir. Karşılaştırmadaki "naif" ham yaml-cpp kodu da bizim yazdığımız bir temsildir; gerçek projelerdeki hatalı kullanımların dağılımını temsil ettiği iddia edilmemektedir.

B. Dış Geçerlilik

Veri Kümesi B, GitHub'daki açık kaynak projelerden toplanmış 647 dosyadan oluşur ve tüm gerçek kullanım biçimlerini temsil etmeyebilir; özel depolardaki, kurum içi üretim

Experiment 6 -- ComposeMorph vs. raw yamll-cpp (careful and naive usage)



Şekil 2. ComposeMorph ile iki ham yamll-cpp kullanımının karşılaştırması (Veri Kümesi B). Solda düzenleme başına değişen satır sayısının medyanı, sağda düzenlenen servisin *diğer* alanlarının korunma oranı. Alt ağacı yeniden inşa eden naif kullanım hiçbir vakada kardeş alanları koruyamamaktadır.

ortamlarındaki dosyalar farklı özellikler taşıyabilir. Korpusun küçük dosyalara eğilimli olması, aynı ekosistem üzerine yapılan daha geniş ölçekli ampirik çalışmalarla uyumludur [2], [22]; yine de 2.000 satırın üzerinde yalnızca iki dosya bulunduğundan, en büyük boyut grubuna ait performans sayıları küçük bir örnekleme dayanmaktadır. Veri Kümesi A bu boşluğu kapatmaz; amacı özellik kapsamını denetlemektir.

C. Yapı Geçerliliği

Değişen satır sayısı, kullanıcının algıladığı koruma kalitesini tam olarak temsil etmez: aynı sayıda satır değişikliği, bir dosyada zararsız bir yeniden girintileme, başka bir dosyada anlam değiştiren bir düzenleme olabilir. Bu riski azaltmak için metinsel ölçümleri iki bağımsız anlamsal oracle ile birlikte raporladık: `docker compose config` ve PyYAML ile yeniden yükleme. Tip değişimi ölçümünde PyYAML'ın YAML 1.1 çözümleme kuralları bir vekil olarak kullanılmıştır; Docker'ın kullandığı `go-yaml` bazı biçimleri farklı çözümlediğinden, PyYAML'a göre tip değiştiren her değer Docker tarafından reddedilmez. Bu ayrım tablolarımızda ayrı sütunlarla verilmiştir.

D. Sürüm Geçerliliği

Sonuçlar belirli sürümlere bağlıdır: yamll-cpp 0.8.0, Docker Compose 2.26.1, GCC 14.2.0 ve ölçüm tarihindeki Compose Specification. Hem spesifikasyon hem de `docker compose` uygulaması zamanla değişmektedir; örneğin şemanın `version` alanına ilişkin katılığı değişirse, Bölüm VIII'daki temel çizgi oranları da değişir. Aynı biçimde, yamll-cpp ileride tırnak bilgisini serileştirmede kullanmaya başlarsa temel çizgi ile ComposeMorph arasındaki fark kapanacaktır. Tüm deneyler tek komutla yeniden koşulabildiği için bu sayılar yeni sürümlerde güncellenebilir.

XI. KISITLAMALAR

Aşağıdaki sınırlar ölçülmüş ve gizlenmemiştir.

- **Yorum satırları korunmaz.** Yorumlar yamll-cpp'nin ayrıştırıcısında veri modeline hiç taşınmadığı için çıktıda bulunmaz. Bu, kütüphanenin düzeltebileceği bir davranış değildir; altta yatan kütüphanede yorum koruma talebi 2015'ten beri açıktır [8].
- **Round-trip bayt düzeyinde özdeş değildir.** Boş satırlar ve girinti tercihleri korunmaz, tek tırnak çift tırnağa dönüşür; dosyaların yalnızca %4,0'ü değişiklik yapılmadan kaydedildiğinde özdeş kalır. Değerlerin YAML tipleri ise korunur.
- **Anchor adları kaybolur.** Anchor/alias ve birleştirme anahtar yapıları genişletilmeden korunur, fakat adlar ayrıştırıcıda saklanmadığı için çıktıda `&1`, `*1` biçiminde yeniden üretilir.
- **Uzun sözdizimi desteği kısımdır.** `environment`, `labels` ve `extra_hosts` için hem kısa hem uzun sözdizimi desteklenir; buna karşılık `Ports`, `Volumes` ve `Networks` yardımcıları liste elemanlarının string olduğunu varsayar. Eşleme biçiminde yazılmış `networks` tanımlarında `Networks::add` hata verir; uzun sözdizimli `ports` ve `volumes` girdilerinde `has/setSource` tip dönüşümü hatasına düşer. Deney 2'deki 32 başarısızlığın tamamı bu sınırdan kaynaklanmaktadır.
- **x- öneksiz bilinmeyen alanlar şemaya aykırıdır.** Kütüphane bu alanları eksiksiz korur, ancak Compose şeması yalnızca x- önekli uzantılara izin verdiğinden dosya `docker compose config` tarafından reddedilir. Bu, kütüphaneden bağımsız bir spesifikasyon kısıtıdır.
- **Doğrulama yüzeyseldir.** Kütüphane içindeki `validate()` yalnızca birkaç iş kuralını denetler;

şema doğrulaması dış araca (docker compose config) bırakılmıştır.

- **Platform.** Geliştirme ve ölçümler Linux üzerinde yapılmıştır; Windows ve macOS'ta test edilmemiştir.

XII. SONUÇ

Docker Compose dosyalarının programatik olarak değiştirilmesi, YAML sözdizimini ayrıştırmaktan ibaret değildir: dosyada aracın modellemediği alanlar, kullanıcı uzantıları, farklı sözdizimi gösterimleri ve anlam taşıyan tırnak tercihleri bulunur. Bu bildiride, bu problemi C++ tarafında ele alan ComposeMorph kütüphanesini sunduk ve çözülebilen ile çözülemeyeni 647 gerçek dünya ve 38 kontrollü Compose dosyası üzerinde yedi deneye ölçtük.

Sonuçlar üç noktada toplanabilir. Birincisi, düzenlemeyi düğüm ağacında yerinde yapmak, modellenmemiş alanların korunması için yeterlidir: bilinmeyen özelliklerin ve `x-*` uzantılarının tamamı korunmuştur. İkincisi, düğüm ağacı tabanlı serileştirme sessiz bir tip değişimi kusuru barındırır; ayrıştırıcının sakladığı tırnak bilgisinin serileştirmede kullanılması bu kusuru ortadan kaldırmış ve Docker doğrulama başarısını %38,1'den %100'e çıkarmıştır; üstelik bunun bir performans bedeli yoktur. Üçüncüsü, koruma davranışı doğru yazılmış ham `yaml-cpp` koduyla da elde edilebilir; tipli API'nin katkısı, doğru olanı varsayılan yol hâline getirmesidir — ölçümlerimizde makul görünen ama hatalı bir ham kullanım, düzenlenen servisin diğer alanlarını hiçbir vakada koruyamamıştır.

Gelecek çalışmalar için üç yön öne çıkmaktadır. Birincisi, yorum ve boşluk korumasını da kapsayan tam CST tabanlı düzenleme; bu, diğer ekosistemlerde çözülmüş olan problemin C++ tarafındaki karşılığıdır. İkincisi, uzun sözdizimi desteğinin `ports`, `volumes` ve `networks` yardımcılara genişletilmesi ve şema farkındalıklı doğrulamanın kütüphane içine taşınması. Üçüncüsü, nesne modelinin Compose Specification ile otomatik eşitlenmesi ve aynı yaklaşımın Kubernetes manifestoları gibi diğer yapılandırma biçimlerine uygulanması.

KAYNAKLAR

- [1] “The Compose specification,” <https://github.com/compose-spec/compose-spec>, 2026.
- [2] M. H. Ibrahim, M. Sayagh ve A. E. Hassan, “A study of how Docker Compose is used to compose multi-component systems,” *Empirical Software Engineering*, vol. 26, 2021.
- [3] J. Beder ve katkıda bulunanlar, “yaml-cpp: A YAML parser and emitter in C++,” <https://github.com/jbeder/yaml-cpp>, 2026.
- [4] A. van der Neut, “ruamel.yaml: YAML 1.2 loader/dumper package for Python,” <https://pypi.org/project/ruamel.yaml/>, 2026.
- [5] E. Aro, “yaml: YAML parser and serialiser for JavaScript,” <https://github.com/eemeli/yaml>, 2026.
- [6] “compose-go: Reference library for parsing and loading Compose YAML files,” <https://github.com/compose-spec/compose-go>, 2026.
- [7] “kyaml: YAML processing for Kubernetes resource configuration,” <https://pkg.go.dev/sigs.k8s.io/kustomize/kyaml>, 2026.
- [8] “option to not eat comments (#154),” <https://github.com/jbeder/yaml-cpp/issues/154>, yaml-cpp hata kayıt sistemi, 2015.
- [9] “Preserve comments after parsing and re-emit them (#457),” <https://github.com/jbeder/yaml-cpp/issues/457>, yaml-cpp hata kayıt sistemi, 2026.
- [10] “libyaml: A C library for parsing and emitting YAML,” <https://github.com/yaml/libyaml>, 2026.
- [11] “Support for emitting and parsing comments (#42),” <https://github.com/yaml/libyaml/issues/42>, libyaml hata kayıt sistemi, 2016.

- [12] P. R. M. J. (biojppm) ve katkıda bulunanlar, “rapidyaml (ryml): A library to parse and emit YAML, fast,” <https://github.com/biojppm/rapidyaml>, 2026.
- [13] “Preserving scalar type information when emitting (#30),” <https://github.com/biojppm/rapidyaml/issues/30>, rapidyaml hata kayıt sistemi, 2020.
- [14] “fkYAML: A C++ header-only YAML library,” <https://fktn-k.github.io/fkYAML/>, 2026.
- [15] “yaml-edit: A formatting-preserving YAML editor for Rust,” https://docs.rs/yaml-edit/latest/yaml_edit/, 2026.
- [16] Instagram Engineering, “LibCST: A concrete syntax tree parser and serializer library for Python,” <https://github.com/Instagram/LibCST>, 2026.
- [17] D. Quirl, “cstree: Concrete syntax tree library,” <https://github.com/domenicquirl/cstree>, 2026.
- [18] “Preserving comments and other human-readable syntax in output yaml (#259),” <https://github.com/kubernetes-sigs/kustomize/issues/259>, kustomize hata kayıt sistemi, 2026.
- [19] T. Tosi, “fishfinger: A Go docker-compose programmatic library,” <https://github.com/TimTosi/fishfinger>, 2026.
- [20] “compose-py: Python library for parsing and loading compose files,” <https://github.com/bonprosoft/compose-py>, 2026.
- [21] “docker-compose-yaml-parser: Parse and modify docker-compose.yaml files,” <https://github.com/WisdomSky/docker-compose-yaml-parser>, 2026.
- [22] K. Eng, A. Hindle ve E. Stroulia, “Patterns of multi-container composition for service orchestration with Docker Compose,” 2023.
- [23] I. Predoia, D. Kolovos, A. Garcia-Dominguez, M. Lenk, W. Ebel ve J. Burkl, “Towards processing YAML documents with model management languages,” *Companion Proceedings of MODELS '24* içinde, Linz, Avusturya, 2024.